High-Level Design (HLD): RAG-Based Customer Support Assistant

1. System Overview

Problem Definition

Customer support teams increasingly face a high volume of repetitive queries that strain human resources and delay response times. While standard chatbots can handle predefined flows, they fail when users ask complex questions requiring specific knowledge base context. There is a need for a system capable of accurately retrieving information from internal documents (e.g., PDFs) and generating concise, contextual answers, while seamlessly escalating to human agents when queries surpass a confidence or complexity threshold.

Scope of the System

This project implements a Retrieval-Augmented Generation (RAG) system with a graph-based orchestration layer. The system will:

1. Parse and chunk knowledge base documents (PDFs).
2. Generate embeddings for text chunks and store them in a vector database.
3. Accept user queries, retrieve relevant context, and construct answer generation prompts.
4. Route queries via a Human-in-the-Loop (HITL) protocol using a graph workflow if automated resolution is deemed insufficient or too risky.

2. Architecture Diagram

```
graph TD
    %% User / Inputs
    User[User Interface<br>CLI/Web]
    DocInput[PDF Knowledge Base]

    %% Ingestion Pipeline
    subgraph Data Flow: Ingestion
        DocInput --> Loader[Document Loader]
        Loader --> Chunks[Text Splitter/Chunker]
        Chunks --> Embed1[Embedding Model]
        Embed1 --> VectorDB[(ChromaDB)]
    end

    %% Query Pipeline
    subgraph Query Flow: Execution (LangGraph)
        User --> |Query| StateGraph[LangGraph State Workflow]
        StateGraph --> IntentNode[Intent & Routing Node]

        IntentNode -->|Complex / Low Confidence| HITL[Human-in-the-loop
        IntentNode -->|Standard Support| RetrieveNode[Retrieval Node]
```

```
    RetrieveNode -->|Query Vector| VectorDB
    VectorDB -->|Top-K Context | RetrieveNode

    RetrieveNode --> GenerationNode[Generation Node<br>LLM Processi
    GenerationNode --> OutputNode[Response Delivery]
    HITL -->|Human Feedback| OutputNode
end

OutputNode --> User
```

3. Component Description

1. **Document Loader**: Extracts text data and metadata from PDF files handling layout and OCR extraction if necessary.
2. **Chunking Strategy**: Splits large documents into smaller, semantically meaningful text chunks (e.g., recursive character splitting with overlap) to maintain topic cohesion and fit LLM context limits.
3. **Embedding Model**: Transforms text chunks into dense vector representations capturing semantic meaning. Open-source models like `all-MiniLM-L6-v2` or OpenAI's `text-embedding-3-small`.
4. **Vector Store**: ChromaDB acts as the local storage and retrieval engine for vector embeddings, offering efficient nearest-neighbor searches.
5. **Retriever**: Queries ChromaDB using the user query's embedding to recall the most relevant *k* document chunks.
6. **LLM**: The core inference engine (e.g., Gemini, OpenAI, or local Llama3) that formulates answers using the combined user prompt and retrieved context.
7. **Graph Workflow Engine**: LangGraph orchestrates the cyclic operations by maintaining state across nodes (Intent, Retrieval, Generation, HITL).
8. **Routing Layer**: A control logic node within LangGraph that evaluates intent. It can perform semantic routing to decide whether to query the DB or jump straight to a human.
9. **HITL Module**: A checkpoint mechanism in LangGraph that halts workflow execution, awaits human review or intervention, and resumes state processing seamlessly.

4. Data Flow

Document Ingestion

1. PDF document is passed to the Document Loader.
2. Text is extracted and split into 500-1000 token chunks with minor overlaps to preserve context boundaries.
3. Each chunk is passed through an Embedding Model to generate a floating-point vector.
4. Chunks and vectors are saved into ChromaDB.

Query Lifecycle

1. User submits a string query.
2. The LangGraph engine captures the query and initiates the workflow state.
3. An Intent Classifier (Agent) determines if the query is conversational, an escalation request, or a support question.
4. If it's a support question, the query is vectorized and compared against ChromaDB records via cosine similarity.

5. The top `N` relevant chunks are retrieved.
6. The query, combined with the retrieved chunks, is structured into a prompt.
7. The LLM generates the answer, which is streamed or returned to the User Interface.
8. If the Intent Classifier detects frustration or complexity, execution halts and is delegated to the HITL node.

5. Technology Choices

- **Vector Database:** *ChromaDB*. Chosen for its lightweight nature, ease of local setup without heavy infrastructure dependencies (like Docker for Milvus/Pinecone), and native integration with LangChain.
- **Workflow Orchestrator:** *LangGraph*. Best suited for building stateful, durable, multi-actor LLM applications. Unlike static chains, LangGraph allows for branching, cyclic loops, and easy implementation of state persistence crucial for HITL.
- **LLM Selection:** *Hugging Face / OpenAI / Gemini* integrations. Flexibility allows swapping the backbone LLM based on cost constraints and compute availability. Open-source models ensure privacy, while managed APIs ensure maximum reasoning capability.

6. Scalability Considerations

- **Handling Large Documents**: Transitioning from local memory vector stores (like naive FAISS) to disk-backed databases like ChromaDB allows millions of embeddings.
- **Query Load Constraints**: Moving the embeddings and generation generation to robust API providers offloads horizontal scalability to managed infrastructure.
- **Latency Strategies**: Introducing caching layers (e.g., Redis or GPTCache) for frequent queries, ensuring the LLM doesn't redundantly process identical support questions.